ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

Комп'ютерні інформаційні мережі

ЛАБОРАТОРНА РОБОТА №10

Виконав:

Ст. Яценко Владислав

ПМІ-33с

Львів - 2025

Тема: Програмування сокетів. Реалізація клієнт-серверного застосунку з послідовним та паралельним режимами обробки клієнтів.

Мета роботи: Отримати практичні навички роботи з мережевими сокетами, навчитися будувати TCP-клієнт/сервер, ознайомитися з принципами багатопотокової обробки та забезпечити багатокрокову взаємодію між сервером і клієнтом.

Хід роботи

У межах лабораторної було розроблено клієнт-серверний застосунок **“Банківський асистент”,** який:

- працює через TCP-сокети;
- має два режими роботи:
  - S — послідовний (сервер обслуговує одного клієнта за раз),
  - P — паралельний (кожен клієнт обробляється в окремому потоці);
- здійснює багатокрокову взаємодію з користувачем:
  - введення імені користувача,
  - введення PIN-коду,
  - вибір операцій у меню;
- реалізує перегляд балансу, поповнення, зняття коштів;
- має розширене логування на сервері (IP, порт, авторизація, транзакції);
- підтримує коректне завершення роботи: при команді STOP сервер повідомляє всіх клієнтів про вимкнення.

Код програми

Код сервера (server.py)

```python
import socket
import threading
import datetime

HOST = "127.0.0.1"
PORT = 9091

USER_DATA = {
    "vlad": {"pin": "1234", "balance": 5000},
    "anna": {"pin": "9999", "balance": 12000}
}

active_clients = []

def log(msg):
    time = datetime.datetime.now().strftime("%H:%M:%S")
    print(f"[{time}] {msg}")

def handle_client(conn, addr):
    active_clients.append(conn)
    log(f"[ПІДКЛЮЧЕННЯ] IP={addr[0]} PORT={addr[1]}")
```

```python
    try:
        while True:
            conn.send("Введіть ім'я користувача: ".encode())
            username = conn.recv(1024).decode().strip()

            if username in USER_DATA:
                conn.send("Користувача знайдено.\n".encode())
                log(f"[OK] Користувач '{username}' підтверджений")
                break
            else:
                conn.send("Користувача не знайдено. Спробуйте ще раз.\n".encode())
                log(f"[WARN] Невідомий користувач '{username}'")

        while True:
            conn.send("Введіть PIN-код: ".encode())
            pin = conn.recv(1024).decode().strip()

            if pin == USER_DATA[username]["pin"]:
                conn.send("Авторизація успішна.\n".encode())
                log(f"[AUTH] Авторизація успішна для '{username}'")
                break
            else:
                conn.send("Невірний PIN. Спробуйте ще раз.\n".encode())
                log(f"[FAIL] Невірний PIN для '{username}'")

        while True:
            menu = (
                "\nОберіть дію:\n"
                "1 - Переглянути баланс\n"
                "2 - Поповнити баланс\n"
                "3 - Зняти кошти\n"
                "4 - Вихід\n> "
            )
            conn.send(menu.encode())
            choice = conn.recv(1024).decode().strip()

            if not choice:
                break

            if choice == "1":
                balance = USER_DATA[username]['balance']
                conn.send(f"Ваш баланс: {balance} грн\n".encode())
                log(f"[INFO] Баланс '{username}': {balance}")

            elif choice == "2":
                conn.send("Введіть суму поповнення: ".encode())
                try:
                    amount = int(conn.recv(1024).decode())
                    USER_DATA[username]['balance'] += amount
                    conn.send("Баланс оновлено.\n".encode())
                    log(f"[DEPOSIT] {username} +{amount}")
                except:
                    conn.send("Некоректне число.\n".encode())

            elif choice == "3":
                conn.send("Скільки зняти? ".encode())
```

```python
79                  try:
80                      amount = int(conn.recv(1024).decode())
81                      if amount ≤ USER_DATA[username]["balance"]:
82                          USER_DATA[username]["balance"] -= amount
83                          conn.send("Операція виконана.\n".encode())
84                          log(f"[WITHDRAW] {username} -{amount}")
85                      else:
86                          conn.send("Недостатньо коштів.\n".encode())
87                  except:
88                      conn.send("Некоректне число.\n".encode())
89
90              elif choice == "4":
91                  conn.send("Вихід із системи.\n".encode())
92                  break
93
94              else:
95                  conn.send("Невірна команда. Спробуйте ще раз.\n".encode())
96
97      except:
98          pass
99
100     finally:
101         if conn in active_clients:
102             active_clients.remove(conn)
103         conn.close()
104         log(f"[СЕСІЮ ЗАВЕРШЕНО] Користувач {addr} відключено.")
105
106
107 def start_server():
108     while True:
109         print("Оберіть режим сервера:")
110         print("S - послідовний")
111         print("P - паралельний")
112
113         mode = input("Введіть режим: ").strip().upper()
114
115         if mode in ("S", "P"):
116             break
117         else:
118             print("Невірний режим. Дозволені лише S або P.\n")
119
120     server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
121     server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
122     server.bind((HOST, PORT))
123     server.listen(5)
124
125     log(f"[СТАРТ] Сервер працює на {HOST}:{PORT}")
126     log("Для зупинки введіть STOP")
127
128     def stop_listener():
129         while True:
130             cmd = input()
131             if cmd.lower() == "stop":
132                 log("[ЗУПИНКА] Сервер відправляє повідомлення всім клієнтам ...")
133
```

```python
            for c in active_clients:
                try:
                    c.send("\nСервер завершує роботу. З'єднання буде закрито.\n".encode())
                    c.close()
                except:
                    pass

            active_clients.clear()

            server.close()
            log("[ГОТОВО] Сервер вимкнено.")
            exit(0)

    threading.Thread(target=stop_listener, daemon=True).start()

    while True:
        try:
            conn, addr = server.accept()
        except OSError:
            break

        if mode == "P":
            thread = threading.Thread(target=handle_client, args=(conn, addr))
            thread.start()
            log(f"[ПОТОКИ] Активних потоків: {threading.active_count() - 1}")
        else:
            handle_client(conn, addr)

if __name__ == "__main__":
    start_server()
```

Код клієнта (client.py)

```python
import socket

HOST = "127.0.0.1"
PORT = 9091

def start_client():
    try:
        client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client.connect((HOST, PORT))
        print("Підключення встановлено.\n")

        while True:
            data = client.recv(1024)
            if not data:
                break

            text = data.decode()

            print(text, end="")

            if "Сервер завершує роботу" in text:
                break

            if text.endswith(": ") or text.endswith("> "):
                user_input = input()
                client.send(user_input.encode())

    except Exception as e:
        print("Помилка:", e)

    finally:
        client.close()
        print("\nЗ'єднання закрито.")

start_client()
```

Структура взаємодії клієнта

1. Клієнт підключається до сервера.
2. Сервер запитує ім'я користувача.
3. Клієнт вводить ім'я → сервер перевіряє.
4. Сервер запитує PIN-код.
5. Після успішної авторизації клієнт отримує меню:
    - 1 — перегляд балансу
    - 2 — поповнення
    - 3 — зняття
    - 4 — вихід
6. Всі помилки вводу не завершують з'єднання — сервер просить повторити.

Результати роботи програми

У цьому розділі наведено поетапну демонстрацію роботи розробленого клієнт-серверного застосунку. Перевірено роботу сервера в обох режимах — послідовному (S) та паралельному (P) — а також взаємодію між кількома клієнтами.

1. Запуск сервера в послідовному режимі (режим S)
    1. У першому терміналі я запустив сервер командою:

```
python server.py
```

Після цього обрав режим **S**

Сервер запустився і почав очікувати клієнтів.

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks> cd Lab10
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python server.py
Оберіть режим сервера:
S - послідовний
P - паралельний
Введіть режим: S
[18:57:29] [СТАРТ] Сервер працює на 127.0.0.1:9091
[18:57:29] Для зупинки введіть STOP
```

2. Підключення першого клієнта (послідовний режим)

У другому терміналі запустив клієнт:

```
python client.py
```

Клієнт успішно підключився до сервера.

При вводі даних яких немає в словнику, передбачено повідомлення про

відсутність такого користувача.
Пройдено авторизацію — введено ім’я та PIN-код.
Після цього з’явилося меню операцій.

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks> cd Lab10
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python client.py
Підключення встановлено.

Введіть ім'я користувача: vl
Користувача не знайдено. Спробуйте ще раз.
Введіть ім'я користувача: vlad
Користувача знайдено.
Введіть PIN-код: 1234
Авторизація успішна.

Оберіть дію:
1 - Переглянути баланс
2 - Поповнити баланс
3 - Зняти кошти
4 - Вихід
>
```

Спроба підключення другого клієнта (послідовний режим)

3. У третьому терміналі запустив другий клієнт:

```
python client.py
```

Оскільки сервер працює в послідовному режимі, другий клієнт не отримав запрошення до введення даних - він очікує, поки перший завершить роботу.

Це підтверджує, що сервер блокує нові з’єднання до завершення попереднього.

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks> cd Lab10
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python client.py
Підключення встановлено.

```

4. Завершення роботи першого клієнта

Після вибору пункту “4 — Вихід” перший клієнт закінчив сеанс.

- сервер автоматично закрив з’єднання першого клієнта;
- другий клієнт негайно отримав запит на введення імені користувача.

```
Оберіть дію:
1 - Переглянути баланс
2 - Поповнити баланс
3 - Зняти кошти
4 - Вихід
> 4
Вихід із системи.

З'єднання закрито.
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10>
```

Клієнт 2 в той час:

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks> cd Lab10
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python client.py
Підключення встановлено.

Введіть ім'я користувача:
```

Звіт по логах на сервері:

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks> cd Lab10
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python server.py
Оберіть режим сервера:
S - послідовний
P - паралельний
Введіть режим: S
[18:57:29] [СТАРТ] Сервер працює на 127.0.0.1:9091
[18:57:29] Для зупинки введіть STOP
[18:58:45] [ПІДКЛЮЧЕННЯ] IP=127.0.0.1 PORT=54185
[18:58:52] [WARN] Невідомий користувач 'vl'
[18:58:54] [OK] Користувач 'vlad' підтверджений
[18:58:56] [AUTH] Авторизація успішна для 'vlad'
[19:02:01] [СЕСІЮ ЗАВЕРШЕНО] Користувач ('127.0.0.1', 54185) відключено.
[19:02:01] [ПІДКЛЮЧЕННЯ] IP=127.0.0.1 PORT=52562
```

2. Запуск сервера в паралельному режимі (режим P)
   1. Після перевірки послідовного режиму я перезапустив сервер і вибрав P:

   У логах сервера з’явилася інформація про кількість потоків.

```
PS C:\Users\igrew\OneDrive\Desktop\UNI\CompInfNetworks\Lab10> python server.py
Оберіть режим сервера:
S - послідовний
P - паралельний
Введіть режим: P
[19:06:31] [СТАРТ] Сервер працює на 127.0.0.1:9091
[19:06:31] Для зупинки введіть STOP
```

   2. Одночасна робота двох клієнтів у паралельному режимі

   У нових терміналах запустив два клієнти:

   Клієнт 1:

```
Введіть ім'я користувача: vlad
Користувача знайдено.
Введіть PIN-код: 1234
Авторизація успішна.

Оберіть дію:
1 - Переглянути баланс
2 - Поповнити баланс
3 - Зняти кошти
4 - Вихід
>
```

Клієнт 2:

```
Введіть ім'я користувача: anna
Користувача знайдено.
Введіть PIN-код: 1234
Невірний PIN. Спробуйте ще раз.
Введіть PIN-код: 9999
Авторизація успішна.

Оберіть дію:
1 - Переглянути баланс
2 - Поповнити баланс
3 - Зняти кошти
4 - Вихід
>
```

На відміну від режиму S:

- обидва клієнти одночасно отримали запрошення на введення імені;
- кожен клієнт працює незалежно від іншого;
- сервер створює окремий потік для кожного клієнта.

У логах сервера з’являються запис:

```
[19:06:31] [СТАРТ] Сервер працює на 127.0.0.1:9091
[19:06:31] Для зупинки введіть STOP
[19:07:33] [ПІДКЛЮЧЕННЯ] IP=127.0.0.1 PORT=63267
[19:07:33] [ПОТОКИ] Активних потоків: 2
[19:07:42] [OK] Користувач 'vlad' підтверджений
[19:07:44] [AUTH] Авторизація успішна для 'vlad'
[19:07:47] [ПІДКЛЮЧЕННЯ] IP=127.0.0.1 PORT=63274
[19:07:47] [ПОТОКИ] Активних потоків: 3
[19:07:49] [OK] Користувач 'anna' підтверджений
[19:07:50] [FAIL] Невірний PIN для 'anna'
[19:07:53] [AUTH] Авторизація успішна для 'anna'
```

3. Перевірка банківських операцій

Для клієнтів виконано декілька операцій:

- перегляд балансу;
- поповнення рахунку;
- зняття коштів;

Сервер коректно відобразив усі транзакції.

```
[19:14:02] [INFO] Баланс 'vlad': 4000
[19:14:18] [DEPOSIT] vlad +5000
[19:14:19] [INFO] Баланс 'vlad': 9000
[19:14:22] [INFO] Баланс 'anna': 12000
[19:14:27] [WITHDRAW] anna -10000
[19:14:29] [INFO] Баланс 'anna': 2000
[19:14:30] [СЕСІЮ ЗАВЕРШЕНО] Користувач ('127.0.0.1', 62146) відключено.
[19:14:33] [СЕСІЮ ЗАВЕРШЕНО] Користувач ('127.0.0.1', 62139) відключено.
```

4. Коректне завершення роботи сервера (STOP)

У першому терміналі ввів команду stop:

```
stop
[19:15:59] [ЗУПИНКА] Сервер відправляє повідомлення всім клієнтам...
[19:15:59] [ГОТОВО] Сервер вимкнено.
```

Висновок:

У ході лабораторної роботи було реалізовано клієнт-серверний застосунок на основі TCP-сокетів із підтримкою послідовного та паралельного режимів обробки клієнтів. Забезпечено багатокрокову взаємодію, авторизацію, роботу меню та логування. Перевірено роботу сервера з кількома клієнтами та коректне завершення роботи за командою STOP. Мета роботи досягнута, принципи програмування сокетів успішно засвоєні.